

PIPELINE OVERVIEW

Three-phase pipeline: raw NLM CDE JSON → cleaned, stripped outputs.

Phase 1: Instruments → **Phase 2: Phrases** →

Phase 3: Branch

Phase	Input	Output
1	allcde.json	inst_stripped.json
2	inst_stripped.json	curated_phrases.tsv
3	allcde.json + patterns	5 stripped JSON variants

PHASE 1: INSTRUMENT PIPELINE

instrument_pipeline.yaml

- 1 mine_instruments — “as part of” patterns
- 2 discover_abbreviations — abbreviated references
- 3 discover_verbatim — verbatim + variant matching
- 4 coalesce_patterns — remove subsumed
- 5 validate_subsumption — empirical per-tinyld
- 6 enrich_fields — add field distribution counts
- 7 **CHECKPOINT — curator reviews TSV**
- 8 strip_instruments — apply curated patterns
- 9 sanity_check — diagnose remaining anchors

Key Outputs

inst_stripped.json	Instruments removed from fields
curated.tsv	Curator-approved patterns
inst_full.tsv / inst_sub.tsv	Strip patterns (full / sub-group)

PHASE 2: PHRASE PIPELINE

phrase_pipeline.yaml

- 1 mine_phrases — k-mer mining (k=25→3)
- 2 discover_verbatim — find verbatim occurrences
- 3 coalesce_patterns — remove subsumed phrases
- 4 field_analysis — add field counts + filtering
- 5 **CHECKPOINT — curator reviews TSV**
- 6 strip_phrases — apply curated patterns
- 7 discovery_report — generate summary

Configuration

Variable	Default	Purpose
k_max	25	Longest phrase
k_min	3	Shortest phrase
min_field_count	1	Min CDEs per field
min_tokens	2	Min tokens

Key Outputs

curated_phrases.tsv	Curator-approved phrases
dedup_phrases.tsv	Whole-text duplicates (> k_max)
verbatim_phrases.tsv	Verbatim surface forms

CURATION WORKFLOW

Human-in-the-loop iterative refinement at each checkpoint:

discover → edit → coalesce → validate
diagnose ← strip ← enrich ↑

Curation Tools

- pattern_util --edit FILE — browser-based TSV editor
- --harvest-to-supplementary — auto-ingest patterns
- --promote-supplementary — local → global config
- workflow resume — continue after checkpoint

PHASE 3: BRANCHING STRIP

branching_strip.yaml — 10 steps, 5 outputs

- 1 strip_inst_full — full instrument removal
- 2 strip_inst_sub — sub-group removal
- 3 expand_temporal — generate temporal variants
- 4 strip_temporal_* — temporal from 3 inputs
- 5 strip_phrase/both_* — curated from 3 inputs
- 6 quality_report — remnant detection

5 Output Branches

inst_full Full instrument removal only
inst_sub Sub-group instrument removal only
phrase Curated phrases only (no instruments)
both_full Full instruments + phrases
both_sub Sub-group instruments + phrases

Split Temporal / Curated

Temporal case-insensitive → **Curated** case-sensitive
Temporal (~2,100 auto) stripped first with **-I**; then curated phrases with verbatim case matching.

QUICK EXECUTION

```
# Phase 1
cde-analyzer workflow run \
  instrument_pipeline.yaml \
  -s input_json=allcde.json

# Phase 2
cde-analyzer workflow run \
  phrase_pipeline.yaml \
  -s input_json=inst_stripped.json

# Phase 3
cde-analyzer workflow run \
  branching_strip.yaml \
  -s input_json=allcde.json \
  -s inst_patterns_base=inst \
  -s phrase_patterns=curated.tsv
```

WORKFLOW ENGINE

YAML-based pipeline orchestrator with checkpoints and resume.

YAML Structure

```
name: my_pipeline
variables:
  input_json: "data.json"
  output_dir: "output/"

steps:
  - name: mine_phrases
    action: phrase_miner
    args:
      input: "${input_json}"
      output: "${output_dir}"
      k-max: 25

  - name: curator_review
    action: checkpoint
    message: "Review patterns"

  - name: strip
    action: strip_phrases
    depends_on: [mine_phrases]
```

Key Features

variables	Template substitution via <code>\${var}</code>
depends_on	Explicit step dependencies
checkpoint	Pause for human curation
--set KEY=VAL	Override variables at runtime
--from-step	Start from a specific step
--dry-run	Validate without executing

CONFIG PRIORITY

Variables resolved in order (last wins):

1. YAML `variables:` block (defaults)
2. Auto-discovered `{output_dir}/{name}_config.yaml`
3. CLI `--set KEY=VALUE` overrides

Auto-discovery: project-local config without modifying shared YAML.

WORKFLOW COMMANDS**Run a Workflow**

```
cde-analyzer workflow run FILE.yaml \
-s input_json=allcde.json \
-s output_dir=phase1_output/ \
-s workers=4
```

Resume After Checkpoint

```
cde-analyzer workflow resume \
--state-file out/.workflow_state.json
```

Check Status

```
cde-analyzer workflow status \
--state-file out/.workflow_state.json -v
```

List & Copy

```
# List available workflows
cde-analyzer workflow list
# Copy template to current directory
cde-analyzer workflow copy \
instrument_pipeline -a my_pipe -d ./
```

Partial Re-run

```
cde-analyzer workflow run FILE.yaml \
--from-step strip_phrases
```

AVAILABLE WORKFLOWS

File	Ph	Steps
instrument_pipeline.yaml	1	9 (mine→strip→sanity)
phrase_pipeline.yaml	2	7 (mine→strip→report)
branching_strip.yaml	3	10 (5-way branch)

COMMON ONE-LINERS

```
# Subset by tinyId
cde-analyzer subset -i d.json \
-l id1,id2 -o sub.json
# Count non-null definitions
cde-analyzer count -i d.json \
--fields definitions.*.definition
# Extract for embeddings
cde-analyzer extract_embed \
-i d.json -o embed.csv
# Merge pattern TSVs
cde-analyzer pattern_util \
-M f1.tsv f2.tsv -o m.tsv
```

RECIPE: DISCOVER & CURATE

```
# 1. Discover patterns
cde-analyzer strip_discover \
-i input.json -p inst.tsv \
-e -o discovered.tsv

# 2. Curate (browser editor)
cde-analyzer pattern_util \
--edit discovered.tsv

# 3. Coalesce (remove subsumed)
cde-analyzer pattern_util \
-c discovered.tsv -o coal.tsv

# 4. Validate empirically
cde-analyzer pattern_util \
-V coal.tsv -i input.json \
-o validated.tsv
```

RECIPE: ENRICH & STRIP

```
# 5. Enrich with field counts
cde-analyzer pattern_util \
-A validated.tsv -i input.json \
-o enriched.tsv

# 6. Strip matching text
cde-analyzer strip_phrases \
-i input.json -p enriched.tsv \
-o stripped.json

# 7. Check residuals
cde-analyzer diagnose_strip \
-i stripped.json \
-o residuals.tsv
```